

Asset Management: Spawning & Despawning

Last Updated for v111

Overview

Every world in Horizon is saved as a **world snapshot**. The snapshot contains the list of shapes, scripts, gizmos, and how they are all organized. When you edit a world in build mode a new snapshot is generated. If you ever make a mistake, you can use the “My Creations Page” (<https://www.facebook.com/FacebookHorizon/creations>) to revert a world back to a previous snapshot.

An **asset** is a collection of objects and scripts that you can save outside of all of your world snapshots. For example, you can make a hat, save it in your Asset Library, and then delete the hat from the world you made it in. In the future you can then add that hat to any world by pulling it out of your Asset Library in build mode. A copy of that asset is then saved in that world’s current snapshot. Note that if you edit the asset in the library, the copy you just created *is not* updated.

Additionally, scripts can load and unload assets into a running instance without the asset needing to be stored in the world’s snapshot. You can have a world that has 1000 outfits (which would likely put your world over capacity if they were all part of the snapshot) by choosing which of them to dynamically load while the world is running in an instance.

To load objects into a running world instance you **spawn an asset**. This loads in a copy of that asset as a collection of new objects and scripts. To unload that copy, you **delete the spawned asset** (“despawning” that copy of the asset).

This feature makes it possible to spawn in items such as scenery, wearables, interactive objects, and much more, but only when you actually need them. For example, a world could unload portions of a map to free up capacity to then load in others. As another example, you don’t need to make 10 copies of a hat; instead, you can make the asset once, and put it in your asset library. Then, without even having any copies in the world snapshot, you can spawn in as many as you need when the world is running (as long as the spawned objects stay under 100% on all capacity meters).

Spawning is similar to placing objects/gizmos/assets from your build menu in build mode except that it does not modify the save-state of the world.

Deleting spawned assets is similar to deleting objects/gizmos/assets in build mode.

Example Uses of Spawning

Spawning static assets - If objects are marked as static (their motion type is “none”) when put into an asset then they will still be static when they spawn. If you spawn in a static asset that

contains emissive objects, then once it spawns in, the objects will cast light into the world. This allows you to load and unload lights without using dynamic lighting. Note though that spawning takes time to happen whereas dynamic lights are instantaneous. You can also spawn in and out of sections of the world as people move around it; this allows you to simulate a much larger world (but only if the visitors stay close enough together). This makes it possible to create **procedural environments and “loading” levels**.

Spawning animated/dynamic assets - When dynamic assets are spawned, the resulting objects will receive the “when world is started event” and start up as usual (e.g. fall due to gravity, auto-play an animation if “play on start” is enabled, etc. You can spawn and despawn dynamic objects as a way to create achievements (e.g. despawn a hat and spawn in a cooler one), to create variety (e.g. choose which car you want to drive or create variance in NPCs), and more. This makes it possible to create **wearable costumes, rewards, “weapon skins”, unlockable items, and so much more**.

Spawning environment gizmos - In published mode spawning in environment blocks with different properties will take effect in your world as soon as they are spawned. If your world has no environment gizmos in it, you can spawn them in and out, changing the sky from “day” to “night” or turning the fog off or on. This makes it possible to create **dynamic environments**.

Spawning Considerations

Keep track of all spawned objects. If you want to delete spawned objects then you need to keep track of them. At minimum, when you spawn in an object you should store it in a script variable or list so that you can reference it in the “delete spawned asset” code block.

Spawned objects should behave the same as non-spawned objects. This includes their limitations. For example, if a spawned object is set as “static”, it can not be manipulated via the codeblocks to move an object. You would need to set it to be an interactive or animated object in order for this to work.

Capacity limits are still there. Even though the spawning occurs in play mode, the capacity limits are in effect. Each asset has a record of how much capacity it needs on each meter (e.g. objects, complexity, vfx, etc). When the asset spawns in, it takes up the capacity on each of those meters. If spawning the asset would take any of the meters over 100% then the spawn will fail. There is currently no way to see how much capacity an asset needs (other than to pull out the asset in build mode and look at the relative change in the meters).

There are two ways to detect a spawn failure - using the global **when an asset spawn fails** event, or by using a special version of the spawn callback event that takes a boolean parameter which indicates whether the spawn was successful (true = success, false = fail).

Asset spawning follows all the same capacity rules as build mode. If you publish a world with 90% *object capacity* and then have that world spawn assets in publish mode, the 90% object capacity will keep increasing until it hits 100% object capacity after which no new assets will spawn. At that point you would need to despawn previously spawned assets to free the object capacity again.

Make assets self-contained. Assets must be entirely contained to work properly (whether imported in build mode or spawning in scripts). This means that all referenced objects and script gizmos attached to objects must be contained within the asset. When an asset is spawned, the scripts are renamed if needed to prevent name clashes (just like when you pull out the asset from the library). It is very important to ensure that you include the scripts and all reference objects in your selection when creating the asset, otherwise your spawned asset will contain broken references.

Be mindful of asset size. Larger assets will take longer to spawn and once they do spawn, the lighting calculation will take longer to settle.

Edit mode vs Visit mode. Edit and Visit mode currently work very differently, and as a result you may see significant differences in how your world behaves with spawning. In particular, spawning performance will be much improved in Visit mode. In addition, bugs that happen in one mode may not appear in the other. Be sure to test your world thoroughly in Visit mode to ensure players are getting the expected experience.

Updating assets used by published worlds. If you modify an asset, new instances of your world will get the updated asset the next time an instance is created. Any already-running world instances will continue to spawn the version that existed when the instance started. There is currently no way to force an already-running instance to refresh / update assets, nor is there a way to delay updating a world to use a new version of an asset until the next publish (i.e. be VERY careful about updating assets in published worlds - you could break it).

Outside of specific exceptions, when an asset despawns, its effects on the world should end. For example, if an asset that was playing audio is despawned, the audio should stop. If a particle gizmo is emitting particles, those particles should disappear. If a projectile launcher has launched a projectile, that projectile should disappear. The one known exception to this is the environment gizmo - when despawning an environment gizmo, the environment will NOT return to the previous state. This is intentional and allows for dynamically changing the environment using spawning.

Limitations and Known Issues (as of May 2023)

There is no way to roll back to previous versions of assets. If you update an asset to a new version, there is no going back. Related to the point above, be very careful about updating an asset in a published world.

Some data can not be included in an asset. For systems such as player persistent variables, leaderboards and achievements which are all defined on the world, spawning an asset that references these will only work if it is spawned in a world in which this data is defined. E.g. It's ok to spawn a leaderboard gizmo that references a leaderboard in a specific world, but if you were to spawn that same asset in another world it would fail.

Sound Recorder gizmos can only be played in the world they were recorded in. Due to a limitation in the way the sound recorder gizmo works, if it is spawned in a world other than the one it was recorded in, it will fail to play. Note that this also applies to dragging this asset out of the asset library into a different world.

Local scripts and spawning. Spawning is no longer supported for local scripts. All spawning must occur in a default script.

Spawned asset ownership. All spawned objects will be owned by the server when they spawn, but can have ownership transferred like any other object. Be aware that transferring ownership will cause the "on world started" script to re-run.

Editing dynamically spawned objects. When in build mode, dynamically spawned objects will appear like any other object. You can view or edit their properties, move them around, etc. However - note that any changes you make will not be saved either to the world or the asset. In addition, modifying a dynamically spawned object could lead to unexpected results when playing the world. In general, it is recommended that you only inspect and do not modify or delete dynamically spawned objects from build mode. If you wish to clear your world of dynamically spawned objects, stopping or resetting the world will accomplish this.

To Spawn Assets

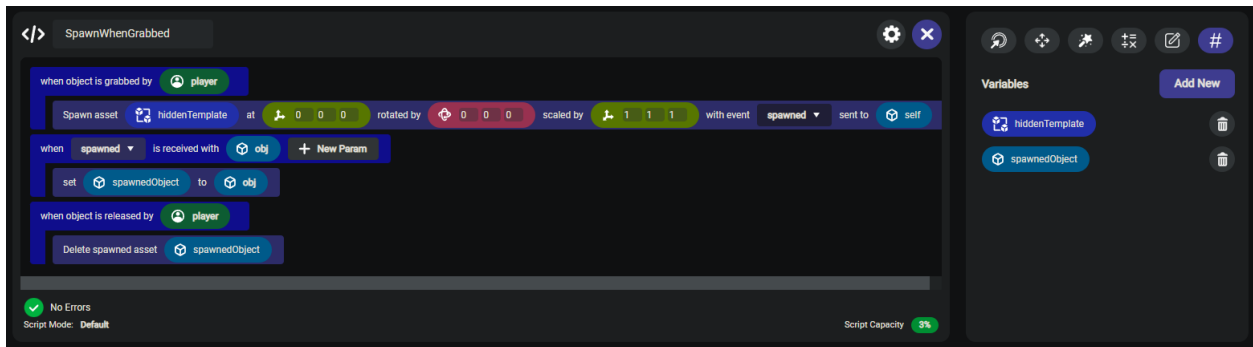
1. **Create an "asset" variable in a script** - To spawn an asset you need a script with an asset variable (which you will set to contain an asset in step 5). You can call this variable whatever you like, e.g. "hatAsset", "hat", "asset", or whatever else you like.
2. **Apply the script to an object** - The script from step 1 needs to be on an object (just like any other script); that object will run the codeblocks that spawn the asset.

3. **Open the scripted object's property Panel** - Open the property panel of the object from step 2; here you will see the Asset Variable field as "empty" in the variables section of the property panel.
4. **Find the asset in Asset Library** - From your build menu navigate to your asset library, then navigate to the asset you want to spawn. Select the view info ("i") icon on that asset.
5. **Connect the reference pill** - On the property panel you opened in step 4, scroll down to see the asset reference pill (blue in color with the asset name){picture}. Select and drag this reference pill to the Asset Variable field "empty" on the objects property panel (Example Object "Obj") where the script was applied in step 3. The field will have the variable name that you chose in step 1. **Note** - *Unlike object references, there is no 'wire' drawn back to the asset in the asset library, just the name of the asset is shown in the variable value field.*
6. **Spawn the Asset** - The script from step 1 can now use the "spawn asset" code block. You use the variable that you created in step 1 to specify what to spawn. **Note** - *you can have one script spawn many different types of assets; you just need to wire them all to different variables in the script.*
7. **(Optional) React to the successful spawn** - Using one of the various methods of being notified about a spawn success (see the section below on Spawn Events and Callbacks), you can get a reference to the root spawned object and do whatever you need to with it (e.g. send a message to the object, or assign it to a variable to later use to despawn the object)

Example: Spawn an object when grabbed

What it does: This example spawns a hidden template whenever a player grabs an object. The template despawns if the player lets go of the object.

How it works: When a person grabs the object the script is on, we spawn the  hiddenTemplate asset and specify that the event  spawned will be sent to  self when the spawn completes. When we receive the  spawned event, we save the spawned object in a variable so that we can despawn it when the object is released.



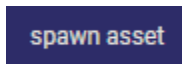
Code Blocks

spawn asset

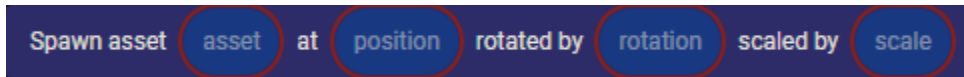
Actions > Spawning > spawn asset

Spawn a new instance of an asset into a running world

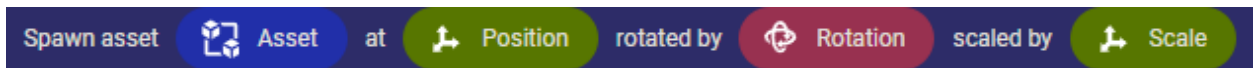
Appearance in Library



Appearance In Composition Pane



Parameters



 Asset	The asset to spawn
 Position 0 0 0	The location the object should be at when it spawns.
 Rotation 0 0 0	The orientation the object should have when it spawns.
 Scale 0 0 0	The scale the object should have when it spawns.

spawn asset with callback

Actions > Spawning > spawn asset with callback

Spawn a new instance of an asset into a running world and fire an event to the target object when the spawn completes. For more details about spawn events, see the section “Spawn Events and Callbacks”

Appearance in Library

spawn asset with callback

Appearance In Composition Pane

Spawn asset **asset** at **position** rotated by **rotation** scaled by **scale** with event **event** sent to **object** with **parameters**

Parameters

Spawn asset **Asset** at **Position** rotated by **Rotation** scaled by **Scale** with event **event** sent to **Object** with **+ New Param**

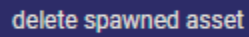
 Asset	The asset to spawn
 Position 0 0 0	The location the object should be at when it spawns.
 Rotation 0 0 0	The orientation the object should have when it spawns.
 Scale 1 1 1	The scale the object should have when it spawns.
event ▼	The callback event. When the object finishes spawning, this event will be sent to the Object variable.
 Object	The receiver object. When the object finishes spawning, the event ▼ will be sent to this object.
+ New Param	Optional additional parameters. These parameters will be sent to the callback event after the object or success parameter.

delete spawned asset

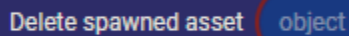
Actions > Spawning > delete spawned asset

Deletes all objects spawned from an asset that was previously spawned, removing the objects and freeing up their capacity

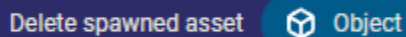
Appearance in Library

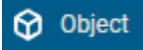


Appearance In Composition Pane



Parameters



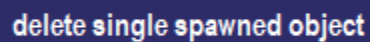
	The object to delete. The object must be the root object created by spawning an asset. For more information on root objects, see the section “Spawned object references”
---	--

delete single spawned object

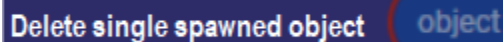
Actions > Spawning > delete single spawned object

Delete an object that was previously spawned, removing it and freeing up the capacity. If there were multiple objects spawned from an asset, only the specified object will be deleted.

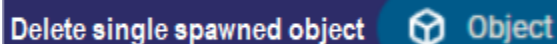
Appearance in Library



Appearance In Composition Pane



Parameters



 Object	The object to delete. The object must be either the object returned from the spawn callback or an object returned from the get spawned objects code block
--	---

get spawned objects

Operators > Spawning > get spawned objects

Given a root object, returns the list of all top level objects spawned with that asset. For more information, see the section “Spawned object references”

Appearance in Library

get spawned objects

Appearance In Composition Pane

get spawned objects from 

Parameters

get spawned objects from  Object

 Object	A root object created by spawning an asset.
--	---

when an asset spawns

Events > Spawning > when an asset spawns

Called any time an asset is spawned successfully. Can be placed in any script.

Appearance in Library

when an asset spawns

Appearance In Composition Pane

when asset spawns  obj  asset

Parameters



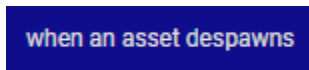
 Object	The root object from the asset which was spawned
 Asset	The asset which was spawned

when an asset despawns

Events > Spawning > when an asset despawns

Called any time an asset is despawned. Will be called after the objects have already been destroyed. Can be placed in any script.

Appearance in Library



Appearance In Composition Pane



Parameters



 Object	The root object from the asset which was despawned
 Asset	The asset which was despawned

when an asset spawn fails

Events > Spawning > when an asset spawn fails

Called any time an asset fails to spawn. This is likely due to exceeding capacity in the world.
Can be placed in any script.

Appearance in Library

when an asset spawn fails

Appearance In Composition Pane

when asset spawn fails  asset

Parameters

when asset spawn fails  asset

 Asset	The asset which was attempting to spawn
---	---

Spawn Events and Callbacks

Spawn Success

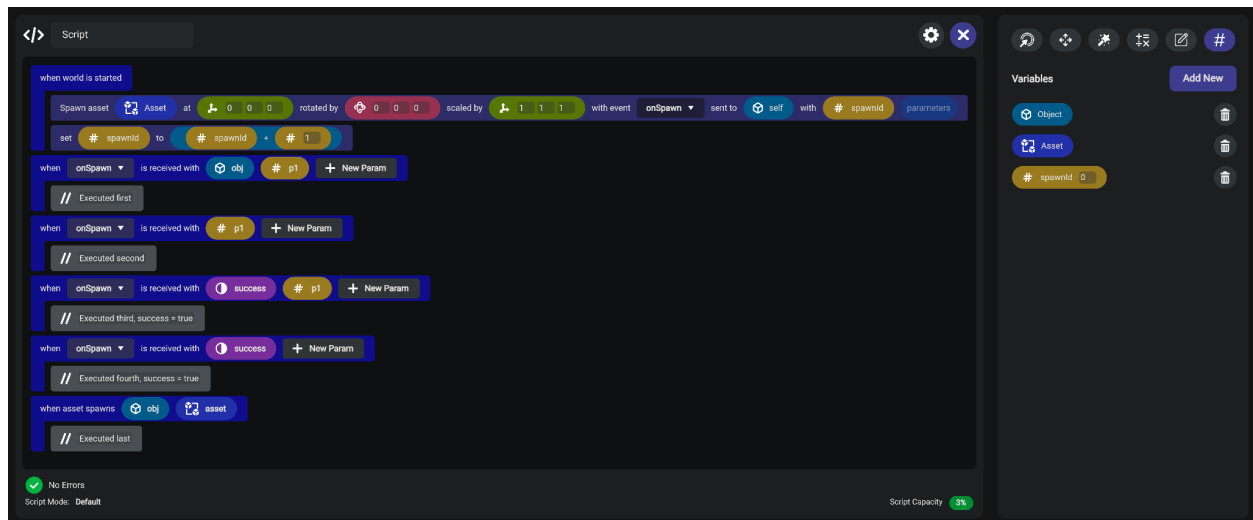
When a spawn completes successfully, a few events will be fired in the following order:

- If using the **spawn asset with callback** code block, the following events will be sent in this order:
 - The callback event specified will be sent with an object parameter followed by all specified additional parameters.
 - The callback event specified will be sent with only the additional parameters (i.e. no object parameter)
 - The callback event specified will be sent with a boolean parameter set to true, followed by the specified additional parameters - this will not be sent if there are no additional parameters
 - The callback event specified will be sent with only a boolean parameter set to true.
- The global **when an asset spawns** event will be sent with a reference to the object spawned and a reference to the asset type of the spawn.
- At the point that the spawn success message has been received, the object is fully ready to be used and it is safe to send messages to the object (note that due to bugs in older versions this was not always true, but this is now fixed).

Example

If the **spawn asset with callback** code block was called and specified to send the event **onSpawn** to **self** with the additional parameter **# spawnid** the following events would be called in this order:

- on onSpawn <obj> <number>
- on onSpawn <number>
- on onSpawn <bool=true> <number>
- on onSpawn <bool=true>
- when an asset spawns <obj> <asset>



Spawn Failure

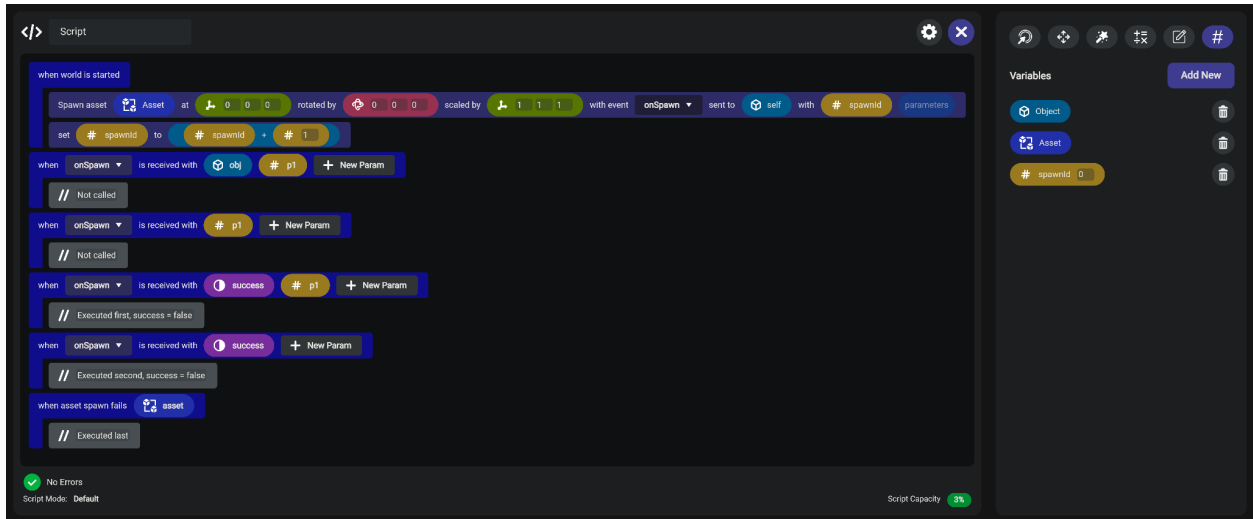
The most likely cause of a spawn failure is that it causes the world to exceed capacity limits (although it is also possible a spawn will fail due to a bug). Regardless of reason, any spawn failure should trigger spawn failure notifications. There are two methods to be notified of a spawn failure, and they will be fired in the following order:

- If using the **spawn asset with callback** code block, the following events will be sent in this order:
 - The callback event specified will be sent with a boolean parameter set to false, followed by the specified additional parameters - this will not be sent if there are no additional parameters
 - The callback event specified will be sent with only a boolean parameter set to false.
- The global **when an asset spawn fails** event will be sent with a reference to the asset type of the spawn

Example

Using the same example as above, if the spawn failed, the following events would be sent in this order:

- on onSpawn <bool=false> <number>
- on onSpawn <bool=false>
- When an asset spawn fails <asset>



Despawn

There is currently only one method to be notified of a despawn:

- The **when an asset despawns** global event will be sent with a reference to the object that was despawned and a reference to the asset type of that spawn.
- Note that this event fires after the object has already been destroyed, so no further operations on that object will be possible from within this event.

Spawned object references

The object reference that is sent via any of the spawn callbacks is referred to as the “root” object of the spawned asset. If the asset contains multiple top level objects (i.e. objects which are not contained inside a group), the first object that was selected when creating the asset will be considered the “root”.

When using the **delete spawned asset** code block, the object reference passed in must be the “root” object. Any attempt to delete another object will fail.

In order to send events to other objects within the asset, the recommended method is to set up the root object to receive all messages and forward them appropriately to other objects within the asset. An alternate method is to use the **get spawned objects** codeblock which takes a “root” object as a parameter and returns a list of all top level objects within the asset.

Note for worlds that used spawning pre-v83:

Prior to v83, there was no concept of a “root” object. The spawn callback would be fired multiple times, once for each top level object in the asset. It was then up to the creator to

manage the list of objects and delete them one by one manually. For compatibility reasons, old versions of these code blocks are still around in worlds which were created prior to v83. These code blocks are marked as [DEPRECATED] and will need to be updated manually to use the new version.

Technical details

- Spawn requests are processed one at a time and in order (i.e. one object will always finish spawning before another one begins). Despawn requests will be processed immediately after the currently processing spawn request finishes (i.e. it will not wait for all pending spawns to complete before despawning), and will also be processed once at a time and in order - however, all despawning objects are immediately made invisible before despawning.
- When an asset is spawned, it is first spawned on the server and then a request is sent to each client to begin the spawning process. The spawn is not considered complete until all clients report back that they have spawned the object successfully - it is at this time that the spawn success callbacks will be fired. This is necessary to ensure that any events sent to the spawned objects will be received on all clients. In the event that a client never responds or takes an unexpectedly long time to spawn, the server will eventually stop waiting and will trigger the spawn success callbacks immediately.
- With the exception of very large assets, the asset data is cached the first time it is downloaded on each client. This results in the subsequent spawns of the same asset being a bit faster than the first. It also means that the version of the asset spawned within a session will always be the same one. There is a limit to the size of this cache, so it is possible that an asset will be re-downloaded on a subsequent spawn (although it will still be the same version of the asset). A world reset will clear this cache. This will allow new versions of the asset to appear in the newly reset instance.

FAQs

Q) My spawns aren't working - what do I do?

A) Check the following things:

- Did you remember to hook up the asset reference on the object? If the asset reference is not set, there should be an error in the script log when you try to spawn it.
- Does the asset still exist? If someone deleted the asset since you hooked it up, it will no longer be available to spawn.
- Double check the position you are spawning to - is it going where you think it should be?
- Double check the scale you are spawning to - is it scaling to size 0?
- Do you have enough capacity to spawn in the object? If the spawn is failing due to capacity, there should be an error about this in the script log.

Q) Can I send events to spawned assets?

A) Yes, but note that the object returned by the spawn callback is for the “root” object of the asset. If you want to send events to another object within the asset, you will either need to have that root object forward the event along or use the **get spawned objects** codeblock to find the object you want.

Q) Can I receive events from spawned assets?

A) Yes, once assets spawn in they act like any other object and can send and receive events. When the asset spawns, you can send events to the spawned object to connect it to other objects in the world. Likewise, the spawned object can interact with triggers, collisions, be grabbed, etc.

Q) My spawn callbacks aren't firing - what did I do wrong?

A) Did you have the correct parameters for your event and does the name of the event match the name specified in the spawn codeblock? Take a look at the “Spawn Events and Callbacks” section for details on how exactly the spawn events need to be constructed.

Q) Can I spawn assets from a non-owner or non-editor's asset library in my world?

A) Yes, once the reference to the asset is made in the asset variables you can spawn in assets from anyone even if they are no longer an editor on your world.

Q) Do spawned objects impact the “dynamics” gauge of the capacity meter?

A) Yes, but only if the assets themselves are dynamic. Spawning and despawning assets is not like “setting visibility” on an object in a script. If an asset is marked as static then spawning it in *will not* impact the dynamics meter; if the object has interaction or physics enabled (or is a text or FX gizmo) then it will.

Q) How does spawning interact with the in-world purchases inventory system?

A) The inventory system makes use of asset spawning in order to spawn the items in the world. The global spawn events (**when an asset spawns**, **when an asset spawn fails**, and **when an asset despawns**) will all fire on inventory spawns and can be used to catch these spawns and have the world react to them. Currently there is no way to know which player triggered the spawn from the inventory, but there is work planned to add additional events specifically for inventory spawns which will contain this information.